uc3m | Universidad Carlos III de Madrid

Grado Universitario en Ingeniería Informática

2025-2026

Trabajo Fin de Grado

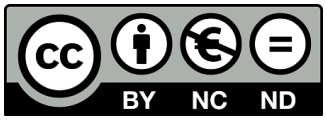
“Sistema Multi-Agente de simulación de autobuses para evaluación de la dotación programada mediante Kpis”

Héctor Álvarez Marcos

Tutor/es

Javier Ignacio Carbó Rubiera

Lugar y fecha de presentación prevista



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**

RESUMEN

RESUMEN RESUMEN RESUMEN

Palabras clave: Agentes Inteligentes, KPI, Simulación de Montecarlo, GAMA

AGRADECIMIENTOS

ÍNDICE GENERAL

1. INTRODUCCIÓN. 1
2. ESTADO DEL ARTE 3
2.1. Agentes Inteligentes 3
2.1.1. Definición 3
2.1.2. Taxonomía 6
2.1.3. Contexto histórico y evolución 8
2.1.4. Comunicación 9
2.1.5. Plataformas de Desarrollo 10
2.1.6. Viabilidad de la solución utilizando Agentes Inteligentes 10
2.2. Simuladores de Transporte 11
2.2.1. Sistemas Multi-Agente como simuladores 11
2.2.2. Simulaciones de Montecarlo 11
2.2.3. Análisis estadístico de datos. 11
3. METODOLOGÍA 12
3.1. Definición del problema 12
3.2. Estructura de trabajo 13
3.3. Herramientas Utilizadas 15
4. ANÁLISIS DE LOS DATOS 17
4.1. Limpieza y preparación 17
4.1.1. Datos geoespaciales 17
4.1.2. Datos de Operación 19
4.2. Estudio y Gaussianidad 22
4.3. Preparación para Simulador 22
5. DISEÑO DE LA SIMULACIÓN 23
6. CASOS DE APLICACIÓN. 24
7. MARCO REGULADOR 25
8. PRESUPUESTO Y COSTES 26
9. TRABAJOS FUTUROS 27

10. CONCLUSIÓN 28
BIBLIOGRAFÍA 29

ÍNDICE DE FIGURAS

2.1 Representación conceptual de un agente y el entorno en el que se encuentra 6
2.2 Evolución histórica del paradigma de agentes inteligentes 9

3.1 Diagrama de Gantt explicativo relativo a las distintas fases del desarrollo . 14

4.1 Visualización en QGIS del mapa geoespacial convertido a shapefile . . . 18
4.2 Visualización en GAMA de los shapefile transformados 18
4.3 Modelo Master:Transaccional de la base de datos 20

ÍNDICE DE TABLAS

2.1 Comparativa de plataformas de desarrollo de sistemas multiagente 10

1. INTRODUCCIÓN

El estudio, gestión y análisis de movilidad urbana en grandes ciudades es uno de los mayores desafíos para las empresas públicas y privadas dentro del sector de transportes. Gracias a la evolución tecnológica en los últimos años, estas empresas se han visto obligadas a adaptarse a los nuevos entornos digitales, buscando nuevos métodos de análisis y cómputo que les permitan mantener una posición más competitiva y obtener resultados mucho más fiables y accesibles en tiempos más cortos. Ejemplos como Pelletier, Trépanier y Morency [1] muestran cómo los datos de las tarjetas inteligentes son usados para el estudio de la movilidad a tres niveles: estratégico, táctico y operacional. Otro ejemplo es el realizado por Nesmachnow y Hernández [2], el cual presenta un estudio hecho en Uruguay sobre implementación de Big Data para analizar el sistema de transporte público de Montevideo. Estos estudios ilustran cómo la adopción de nuevas tecnologías es un hecho creciente en el ámbito del transporte.

En el contexto del problema se sitúa la Empresa Municipal de Transportes de Madrid (EMT)[1], operador del servicio de autobuses urbanos de la capital española. En los últimos años, ha impulsado múltiples iniciativas de innovación tecnológica, destacando «Madrid Green Urban Mobility Lab» o la inclusión del sistema «Galileo». La primera es una iniciativa sin ánimo de lucro que busca promover la movilidad urbana sostenible[2]. «Galileo», por su parte, es un sistema europeo de navegación por satélite, incorporado en la totalidad de la flota con el fin de mejorar la precisión del posicionamiento de los buses[3]. En el contexto de este trabajo, se busca mejorar el análisis del servicio prestado para sus líneas de autobús. Para ello, la Empresa Municipal plantea validar la viabilidad de construir una herramienta que permita extraer la información relevante sobre el desempeño de la flota de autobuses a lo largo de una línea específica.

En este trabajo se propone una implementación de software basada en agentes inteligentes que permite simular el comportamiento de los de autobuses para una línea concreta. Este sistema permite extraer distintas métricas de rendimiento (KPI, del inglés *Key Performance Indicators*). El simulador recibe distintos parámetros de entrada derivados de un estudio realizado sobre datos reales de operación de una línea a lo largo de diez meses de servicio. Esta base permite al usuario final del sistema evaluar estos escenarios a partir de evidencias empíricas, con el objetivo último de otorgar una herramienta que pueda servir

[1] https://www.emtmadrid.es(accedido: 6 de mayo de 2026)

[2] EMT Madrid, «*Madrid Green Urban Mobility Lab* convertirá a la capital en un referente internacional en innovación y tecnología de la movilidad», 26 de septiembre de 2022. Disponible en: https://www.emtmadrid.es/Noticias/%E2%80%98Madrid-Green-Urban-Mobility-Lab%E2%80%99-convertira-a-la.aspx(accedido: 6 de mayo de 2026).

[3] EU Agency for the Space Programme (EUSPA), «EMT Madrid announces readiness to Galileo in all the metropolitan bus fleet», 19 de abril de 2019. Disponible en: https://www.euspa.europa.eu/newsroom-events/news-archive/emt-madrid-announces-readiness-galileo-all-metropolitan-bus-fleet(accedido: 6 de mayo de 2026).

para mejorar la eficiencia en la planificación diaria de autobuses. Para facilitar la usabilidad del sistema, se ha levantado el simulador sobre una interfaz web, que permite tanto la personalización de las simulaciones, como la visualización de las distintas estadísticas resultantes.

A lo largo de este documento, se presentan, discuten y valoran las distintas decisiones de diseño, así como los retos enfrentados, y las conclusiones obtenidas del trabajo. En el capítulo segundo del documento, se discute acerca del estado de la cuestión, en el que se analizan diversas soluciones existentes en la literatura acerca del problema y cómo han ido evolucionando los distintos conceptos clave a lo largo del tiempo. Seguidamente, el capítulo tercero trata la metodología de trabajo escogida, junto a la explicación detallada sobre cada uno de los pasos. COMPLETARË A LA QUE SIGA AÑADIENDO COSAS A LA MEMORIA

2. ESTADO DEL ARTE

En este capítulo se expone una revisión analítica de la literatura existente relevante para el problema abordado en este trabajo. La revisión se organiza en torno a los principales ejes del desarrollo de la solución, ahondando en cómo se resuelven los distintos problemas existentes en cada eje y cómo se diferencia la implementación realizada.

2.1. Agentes Inteligentes

2.1.1. Definición

Existen muchas acepciones de lo que es un agente inteligente dentro de la literatura, entre ellas, una de las más citadas, Maes [3, p. 139], los define como «Los agentes autónomos son sistemas computacionales que habitan en un ambiente dinámico complejo, sienten y actúan autónomamente en ese entorno y al hacerlo llevan a cabo un conjunto de objetivos o tareas para las que fueron diseñados.»[4]

Otra definición muy citada en la literatura y ampliamente reconocida es de Jennings y Wooldridge [4, p. 4], «Un agente es un sistema computacional situado en un entorno, y que es capaz de realizar acciones autónomas en este con el fin de cumplir sus objetivos de diseño»[5]

Ambas definiciones comparten puntos comunes, como la especificación de que un agente es un **sistema computacional** y que buscan cumplir una serie de objetivos y tareas para las que fueron diseñados[3], [4]. Dentro de los distintos conceptos a discutir sobre agentes, se pueden subdividir estos dentro de dos grupos principales: **Características del Entorno** y **Características propias del elemento agente**. En la figura 2.1 se muestra una representación gráfica de los conceptos principales de cada uno de los dos grupos. A continuación se exponen y discuten las diferentes características de estos.

- **Capacidades del entorno**

  La definición formal de Jennings y Wooldridge no especifica características propias del entorno, pero en párrafos posteriores, rechaza la hipótesis de llamar agentes inteligentes a aquellos que se desarrollen en escenarios que no requieran **flexibilidad** y **complejidad**, utilizando como ejemplo comparativo el sistema autónomo de un termostato[4]. Por otra parte, Maes [3] sí que explícitamente caracteriza el ambiente

[4]Traducción del original: «*Autonomous agents are computational systems that inhabit some complex, dynamic environment, sense and act autonomously in this environment, and by doing so realize a set of goals or tasks for which they are designed*».

[5]Traducción del original: «*an agent is a computer system situated in some environment, and that is capable of autonomous action in this environment in order to meet its design objectives*».

como uno dinámico y complejo dentro de la definición de agente. Esto lleva a poder remarcar que no es adecuado, relativo a cuestiones meramente formales, referirse a un sistema que no tiene un entorno caracterizado por su dinamismo y complejidad como un sistema apto para incluir una solución basada en agentes inteligentes.

Otro tema que se define como base que es imperioso remarcar en la revisión literaria es el de la **capacidad distribuida del conocimiento**. Tanto Maes, que especifica que los agentes son creados como pequeños expertos en un tema concreto y que se crean de forma distribuida[3], como Jennings y Wooldridge, los cuales mencionan que una característica que se puede tener en consideración del dominio en el cual se puede aplicar una solución basada en agentes es la existencia del conocimiento de forma distribuida[4]. Al explorar otros artículos en los que se mencione dicha característica, como puede ser la revisión literaria sobre agentes realizada por Nwana, se observa que se incluye como una de las motivaciones para construir un sistema de agentes el solucionar problemas inherentemente distribuidos[5]. Esto permite trazar un consenso generalizado sobre esta característica de los dominios en los que los agentes trabajan.

- **Características propias del elemento agente**.

  Con respecto a la caracterización propia de los agentes, resulta necesario analizar de forma sistemática las distintas propiedades que los definen. A continuación, se presentan las principales características basadas en la arquitectura de los agentes de Wooldridge y Jennings [6], detallando cada una de ellas:

  - **Autonomía**: Discutida ampliamente por Castelfranchi [7] en 1995, incluye varias cuestiones a destacar. Primeramente, una definición débil de autonomía puede postularse como la capacidad de completar una tarea u objetivo concreto sin necesidad de interacción humana[7]. Para fortalecer la definición, resulta imperioso profundizar en las diferentes características necesarias para considerar autonomía real, ya que un concepto de autonomía tan simple como el autocompletado de tareas sin interacción da paso a posibles malinterpretaciones. El mismo Castelfranchi presenta un concepto como el de «Autonomía ejecutiva» para referirse a aquella autonomía que carece de capacidad de toma de decisiones, que se limita a seguir una serie de instrucciones u órdenes sin preferencias personales [7]. Esta subcategoría permite encapsular y diferenciar la autonomía a dos niveles. En la capa inferior está esa «autonomía ejecutiva», que permite al agente resolver tareas sin intervencionismo humano, pero para presentar completa autonomía, hace falta una capa superior que presente «racionalidad». Esta hace referencia a la capacidad de los agentes de tomar decisiones de adopción o rechazo de ideas basada en sus objetivos. Se especifica que ambas ideas deben coexistir para que haya **autonomía completa**. La falta de alguna de ellas reduce las capacidades autónomas del agente en cuestión.

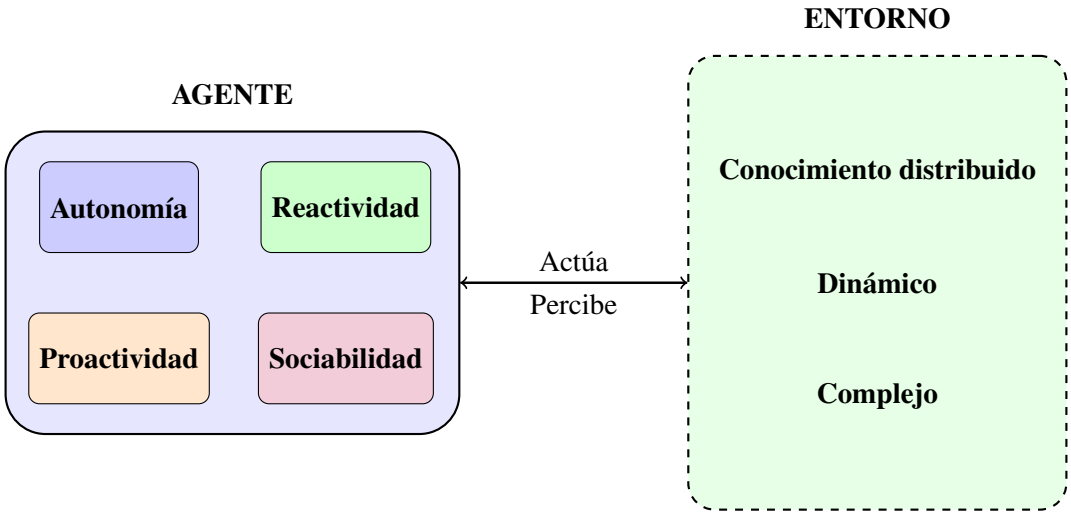
- **Reactividad**: La reactividad se define como la capacidad de un agente de percibir su entorno y responder de forma oportuna a los cambios que se producen en él [6]. Esta propiedad implica que el agente no opera de forma aislada respecto a su entorno, sino que mantiene una vinculación continua con él: percibe estímulos y genera respuestas adecuadas en función de las condiciones observadas. El fundamento teórico de la reactividad tiene su origen en el trabajo de Brooks [8], quien propuso la denominada *arquitectura de subsunción* (*subsumption architecture*). En dicha arquitectura, el comportamiento inteligente emerge de la interacción de capas de control simples con el entorno, sin necesidad de mantener un modelo simbólico interno del mundo. Cada capa vincula directamente la percepción con la acción mediante reglas estímulo-respuesta, lo que permite al sistema operar en entornos dinámicos con tiempos de respuesta reducidos [8]. Este enfoque supuso una ruptura con los paradigmas deliberativos de la inteligencia artificial clásica, en los que la toma de decisiones dependía de un proceso de razonamiento simbólico previo a la acción. No obstante, como señalan Wooldridge y Jennings [6], la reactividad por sí sola no es suficiente para caracterizar a un agente inteligente, ya que un sistema puramente reactivo carece de la capacidad de perseguir objetivos a largo plazo, lo que motiva la necesidad de la proactividad como propiedad complementaria.
- **Sociabilidad**: El término *sociabilidad*, definido por la RAE como «cualidad de sociable»[6], remite al concepto de *sociable*, que a su vez se define como «naturalmente inclinado al trato y relación con las personas o que gusta de ello»[7]. En el contexto de los sistemas multiagente, la sociabilidad se define como la capacidad de comunicación entre agentes mediante un lenguaje determinado[6], y representa el habilitador de la cooperación entre agentes. Según Nwana [5], esta cooperación es la razón de ser de la construcción de sistemas con múltiples agentes. Existen diferentes formas de comunicación, explicadas en profundidad en la sección 2.1.4.
- **Proactividad**: Esta propiedad se refiere a la capacidad de tomar la iniciativa y exhibir un comportamiento dirigido a objetivos [6, p. 116]. Un agente proactivo no se limita a reaccionar ante estímulos externos, sino que es capaz de actuar por iniciativa propia con el fin de satisfacer sus objetivos de diseño. Esta distinción es relevante porque, como señalan Wooldridge y Jennings [6], lograr un equilibrio adecuado entre comportamiento reactivo y proactivo constituye uno de los principales desafíos en el diseño de agentes inteligentes: un agente excesivamente reactivo puede ser incapaz de mantener un plan de acción coherente ante perturbaciones continuas del entorno, mientras que un agente excesivamente proactivo puede ignorar cambios relevantes en su en-

[6]Real Academia Española, *Diccionario de la lengua española*, 23.ª ed. https://dle.rae.es/sociabilidad(accedido: 6 de mayo de 2026)

[7]Real Academia Española, *Diccionario de la lengua española*, 23.ª ed. https://dle.rae.es/sociable(accedido: 6 de mayo de 2026)

torno por perseguir objetivos que han dejado de ser válidos. Franklin y Graesser [9] recogen esta propiedad bajo la denominación de *goal-oriented*, definiéndola como la capacidad del agente de no limitarse a actuar en respuesta al entorno, sino de perseguir activamente sus propios objetivos. En la práctica, la proactividad se manifiesta en la capacidad del agente de planificar, anticipar situaciones futuras o iniciar acciones sin que un estímulo externo inmediato las desencadene.

Además de estas cuatro propiedades, la literatura identifica otras características que definen subclases más específicas de los agentes, como la capacidad de aprendizaje, la movilidad o la continuidad temporal [9]. No obstante, las cuatro propiedades expuestas constituyen un núcleo comúnmente aceptado para la caracterización de agentes inteligentes [6].



Fuente: elaboración propia a partir de [3], [4], [6].

Fig. 2.1. Representación conceptual de un agente y el entorno en el que se encuentra

2.1.2. Taxonomía

En cuanto a la clasificación de los distintos tipos de agentes se pueden encontrar dos principales en la literatura.

La primera, basada en la arquitectura del sistema construido, presentada por Wooldridge y Jennings [6], expone tres principales pilares arquitectónicos para la clasificación de agentes:

- **Arquitectura Deliberativa**: Basada en los paradigmas de la IA simbólica, establece la construcción para cada agente de un modelo simbólico del mundo, en el que las decisiones se toman mediante razonamiento lógico derivado de esta concepción[6].

Esto genera un problema principal, cómo se traslada ese comportamiento del mundo real a un modelo simbólico. No existe un consenso generalizado, una fórmula que permita realizar esta proyección, sin embargo, existen varias implementaciones que abrazan este concepto de *arquitectura deliberativa*. La más popular se conoce como «arquitectura BDI»(**B**eliefs **D**esires **I**ntentions), fue formulada por Bratman [10] en 1987. Esta, construye un modelo simbólico del mundo real (Beliefs), que junto a una serie de objetivos a resolver (Desires), y unos planes seleccionados de la interacción del agente con el entorno (Intentions) crean un modelo de *agente inteligente* deliberativo

- **Arquitectura Reactiva**: Basada en el concepto de reactividad mencionado anteriormente, busca desligarse de la IA simbólica que caracteriza a la *Arquitectura deliberativa*. Brooks[11]-[13] propone que la inteligencia no tiene por qué presentarse como una representación simbólica, en contra de lo que la arquitectura deliberativa establece. Plantea que la inteligencia está situada en el mundo, y no fragmentada en sistemas expertos, generándose también por la interacción entre el agente y el entorno.

- **Arquitectura híbrida**: Propone un enfoque mixto que combina un subsistema reactivo, encargado de responder de forma inmediata a los cambios del entorno, con un subsistema deliberativo, responsable de la planificación y el razonamiento a largo plazo [6]. Entre las propuestas más influyentes se encuentra la arquitectura *TouringMachines* de Ferguson [14], basada en tres capas horizontales —reactiva, de planificación y de modelado— en las que cada capa tiene acceso independiente a la percepción y a la acción. Un enfoque alternativo es la arquitectura *InteRRaP* de Müller, Pischel y Thiel [15], que emplea una estructura vertical por capas en la que únicamente la capa inferior percibe el entorno y únicamente la superior ejecuta acciones, reduciendo la competencia entre capas a costa de una mayor rigidez en el flujo de control. El objetivo de este tipo de arquitecturas es combinar la rapidez de respuesta del nivel reactivo con la capacidad de planificación del nivel deliberativo, buscando el equilibrio entre ambas propiedades que Wooldridge y Jennings [6] identifican como uno de los principales desafíos en el diseño de agentes inteligentes.

El segundo enfoque, enunciado por Franklin y Graesser [9], consiste en una clasificación inspirada en la taxonomía biológica, organizada como un árbol jerárquico en el que cada nivel representa una distinción basada en la naturaleza del agente. En el nivel superior, los agentes autónomos se clasifican en tres categorías: **biológicos** (seres humanos, animales), **robóticos** (robots físicos autónomos) y **computacionales** (entidades software). Los agentes computacionales se subdividen a su vez en **agentes software** orientados a tareas específicas, entretenimiento o seguridad y **agentes de vida artificial**, que habitan entornos simulados. Adicionalmente, los autores proponen que, independientemente de su posición en el árbol, los agentes pueden clasificarse según el **subconjunto de**

propiedades que exhiben —reactividad, autonomía, orientación a objetivos, continuidad temporal, sociabilidad, aprendizaje, movilidad, flexibilidad y personalidad—, de manera que la adición de propiedades define subclases cada vez más específicas [9]. Este enfoque complementa la clasificación arquitectónica de Wooldridge y Jennings [6], ya que permite caracterizar a un agente tanto por *cómo está construido* (arquitectura) como por *qué tipo de entidad es* (naturaleza) y *qué propiedades exhibe* (capacidades).

En la justificación de la arquitectura de los agentes escogida, se tiene en cuenta la taxonomía presentada por Wooldridge y Jennings [6], la cual permite identificar el tipo de arquitectura más adecuado en función de las características del dominio y de los requisitos del problema. Dicha justificación se desarrolla en el capítulo de diseño del sistema.

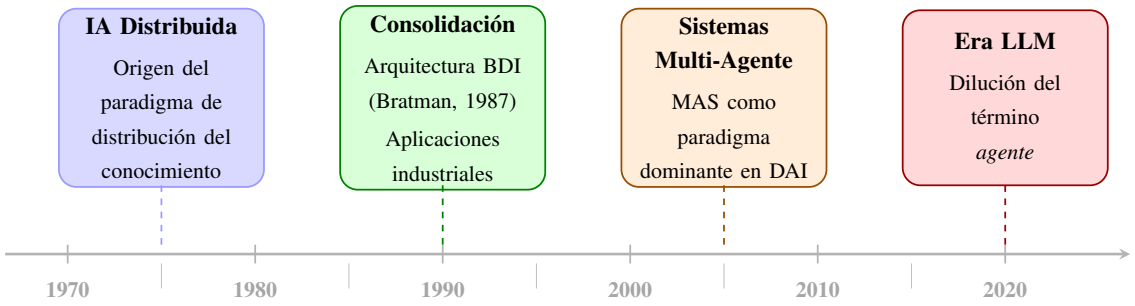
2.1.3. Contexto histórico y evolución

Este concepto de agente inteligente surge como derivado de la **Inteligencia Artificial Distribuida**. La IA distribuida, cuyo origen se remonta a los años 70/80, aparece en respuesta a la falta de socialidad de la ciencia cognitiva. Este nuevo paradigma de distribución del conocimiento trata de romper lo establecido hasta el momento y considerar el entorno como un escenario donde distintos elementos conviven y se relacionan [16].

Ya en la década de los 90, el «agente inteligente» empieza a consolidarse. Se presentan arquitecturas como la **BDI** presentada por Bratman [10] y discutida anteriormente, y empiezan a aplicarse los agentes a soluciones reales, como la implementación de sistemas autónomos en líneas de producción de bobinas de acero, o la gestión del tráfico del tren bala en Japón [17].

Durante los 2000, los sistemas Multi-Agente (o **MAS**) se establecen como el principal paradigma dentro de la Inteligencia artificial Distribuida, absorbiendo progresivamente la terminología y los objetivos de investigación que anteriormente se agrupaban bajo la denominación genérica de DAI [18].

En la actualidad, la llegada de los LLM (Large Language Models) ha supuesto una revolución en el ámbito de la **Inteligencia Artificial**, y el campo de los «Agentes inteligentes» también se ha visto afectado. Un estudio realizado por Botti [19], expone cómo estos nuevos modelos de lenguaje han expandido el concepto de *agente* al mundo y cómo eso ha generado una malinterpretación del término que se lleva discutiendo en la literatura todos estos años, comparando las distintas diferencias que existen entre el término clásico y la moda «Agéntica». En el presente trabajo, el término *agente* se emplea exclusivamente en su acepción clásica, conforme a las definiciones de Maes [3] y Jennings y Wooldridge [4] expuestas al inicio de esta sección.



Fuente: elaboración propia a partir de [10], [16]-[18], [20]

Fig. 2.2. Evolución histórica del paradigma de agentes inteligentes

2.1.4. Comunicación

Como ya se ha mencionado, uno de los conceptos clave de los agentes es el de sociabilidad, concepto que se presenta como habilitador necesario para la cooperación interagente. Es mandatorio establecer unos criterios para la correcta comunicación y entendimiento entre interlocutores. En respuesta a esto, a lo largo de la existencia de los agentes se han ido desarrollando unos estándares de comunicación para el correcto comportamiento de las comunicaciones.

El primero surge en 1993, denominado **KQML** (*Knowledge Query and Manipulation Language*) y desarrollado en el marco del programa *Knowledge Sharing Effort* de DARPA a principios de la década de 1990 [21]. KQML definía un formato de mensaje y un protocolo para el intercambio de conocimiento entre agentes, organizando cada mensaje en tres capas: contenido, comunicación y mensaje. Pese a su amplia adopción en la investigación inicial sobre sistemas multiagente, KQML presentaba limitaciones relevantes: la existencia de múltiples implementaciones incompatibles entre sí, y la ausencia de una semántica formal para sus performativos, cuyo significado se describía de forma informal [21].

En respuesta a estas limitaciones, en 1996 se fundó la *Foundation for Intelligent Physical Agents* (**FIPA**), una organización dedicada a la creación de estándares de interoperabilidad para sistemas basados en agentes. FIPA publicó entre 1997 y 2002 un conjunto de especificaciones que incluyen un lenguaje de comunicación propio (**FIPA-ACL**) [20], una biblioteca de actos comunicativos con semántica formal basada en lógica modal BDI [22], y un conjunto de protocolos de interacción que regulan la estructura de las conversaciones entre agentes como FIPA-Query y FIPA-Request. Estas especificaciones abordaron las principales debilidades de KQML, proporcionando una semántica formal, un conjunto estandarizado de performativos y protocolos de interacción predefinidos.

En 2005, FIPA fue aceptada como comité de estándares del *IEEE Computer Society*, bajo cuyo auspicio se mantienen actualmente las especificaciones. Si bien estas no constituyen estándares IEEE numerados en sentido formal, son consideradas el estándar de facto para la comunicación entre agentes en sistemas multiagente. El estándar FIPA es el elegido para el desarrollo de las distintas interlocuciones entre los agentes.

2.1.5. Plataformas de Desarrollo

El auge de los agentes supuso la creación de nuevos lenguajes y plataformas que permitieran la explotación de este nuevo paradigma. Múltiples soluciones han sido desarrolladas para satisfacer estas necesidades. Mayoritariamente surgieron entre finales de los 90 y principios de los 2000, cuando MAS se establece como estándar dentro de la DAI. Entre ellas se destacan JADE[23], NetLogo[24], MASON[25], Jason[26] y GAMA[27]. En la tabla 2.1, se muestra una comparativa de estas plataformas.

TABLA 2.1. COMPARATIVA DE PLATAFORMAS DE DESARROLLO DE SISTEMAS MULTIAGENTE

Característica	**JADE**	**NetLogo**	**MASON**	**Jason**	**GAMA**
Año	1999	1999	2005	2007	2007
Tipo	Middleware	ABM	ABM	BDI	ABM
Lenguaje	Java	NetLogo	Java	AgentSpeak	GAML
FIPA	Sí	No	No	No	Sí
GIS	No	Limitado	No	No	Sí
Modo batch	No	Sí	Sí	No	Sí
Escalabilidad	Media	Baja	Alta	Baja	Alta
Curva de aprendizaje	Media	Baja	Alta	Alta	Baja
Visualización	Limitada	Sí	Sí	No	Sí
Código abierto	Sí	Sí	Sí	Sí	Sí

Fuente: elaboración propia a partir de [23]-[27].

Como elección se ha escogido GAMA[27], una plataforma de código abierto, con lenguaje propio y construida sobre Java. Se escoge esta plataforma por tres motivos principales, la inclusión de FIPA[20] como estándar de comunicaciones, la integración con mapas geoespaciales y la facilidad de escalado. El incluir ya un estándar para la comunicación facilita su implementación dentro de la solución final, y debido a la naturaleza de los datos del problema, la integración de mapas GIS (Geographic information system) facilita la lectura de los datos de las rutas de los autobuses. Por último, el escalado está facilitado por el modo batch, que permite múltiples ejecuciones paralelas y un despliegue del código por línea de comandos. Estos motivos hacen que GAMA[27] se presente como plataforma idónea para este problema.

2.1.6. Viabilidad de la solución utilizando Agentes Inteligentes

En el contexto del problema a resolver tratado a lo largo de este documento, resulta necesario validar que el dominio existente en el que se encuentra sea el adecuado basándose en las características formales discutidas a lo largo de este apartado. Concretamente,

una de las principales características del problema se basa en la variación de oferta y demanda de pasajeros a lo largo de un conjunto de paradas durante un día concreto. Este entorno se presenta altamente **dinámico** y **complejo**, ya que existen multitud de factores que aleatorizan y varían el rango de soluciones, como la demanda variable dependiente de los cambios de hora. Por otra parte, el conocimiento relevante para el problema se encuentra **distribuido** entre múltiples puntos del sistema: la información sobre la demanda de pasajeros varía según la ubicación geográfica y la franja horaria, los tiempos de viaje dependen de tramos específicos del recorrido, y la disponibilidad operativa fluctúa de forma independiente en cada vehículo.

La verificación de que los agentes implementados satisfacen las propiedades discutidas —autonomía, reactividad, sociabilidad y proactividad— se aborda en el capítulo de diseño.

Es preciso concluir, por tanto,basándose en la literatura en el ámbito relativo al dominio, que una solución basada en agentes inteligentes resulta metodológicamente válida para el problema, satisfaciendo las condiciones discutidas con anterioridad.

2.2. Simuladores de Transporte

La simulación aborda un problema fundamental dentro de cualquier ámbito, ser capaz de recrear un escenario concreto con el fin de extraer o detectar posibles anomalías de una forma temprana. En el contexto del transporte urbano, validar las capacidades de absorción de la congestión en horas puntas resulta clave, al presentarse este como un problema crucial [28].

2.2.1. Sistemas Multi-Agente como simuladores

2.2.2. Simulaciones de Montecarlo

2.2.3. Análisis estadístico de datos

3. METODOLOGÍA

En este capítulo se explican de forma precisa y justificada los distintos métodos seguidos en este problema, así como la presentación de un esquema del desarrollo. Para ello, se expone primeramente la contextualización avanzada del problema en cuestión a resolver, seguido de la explicación de las distintas etapas que constituyen el trabajo. Posteriormente, se mencionan las distintas herramientas que se usan en este proyecto.

3.1. Definición del problema

Este trabajo se realiza dentro de la Cátedra *Inteligencia Artificial para el Transporte y Movilidad*[8]. Esta establece un nexo colaborativo entre la EMT y la Universidad a través del grupo de investigación GIAA[9]. Como se explica de forma breve en la introducción, La Empresa Municipal de Transportes propone estudiar la viabilidad de la construcción de un sistema para analizar el rendimiento de rutas de autobuses. El principal motivo detrás de este planteamiento es la falta de un simulador propio para realizar esta tarea, que se suma a la existencia de una entidad superior a la EMT, el CRTM(Consorcio Regional de Transportes de Madrid) [10], que se encarga de validar diariamente las dotaciones propuestas por la compañía de autobuses. La «caja negra» que implica este paso para la validación supone un componente extra que justifica esta propuesta. La realización de este proyecto se ha enmarcado en un trabajo de investigación de una duración de ocho meses a lo largo de los cuales se ha ido desarrollando la solución planteada, bajo la supervisión bisemanal por parte de la EMT mediante reuniones telemáticas.

En una fase de desarrollo inicial, se propuso por parte de la EMT la evaluación de las cinco líneas *express* de la compañía, al ser líneas con un número de paradas bajo relativo al general de las rutas. Esta distinción permite acotar el análisis previo a la simulación, necesario para la validación de la viabilidad del simulador.

Para la utilización de datos reales, la EMT puso a disposición una base de datos relacional con los datos de operación de las paradas de las líneas Express desde el 1 de julio de 2025, con actualizaciones diarias, permitiendo ir ampliando la base que compone el análisis a medida que se iba completando.

[8] La UC3M y EMT Madrid presentan una nueva Cátedra en Inteligencia Artificial para el transporte y la movilidad https://www.uc3m.es/ss/Satellite/UC3MInstitucional/es/Detalle/Comunicacion_C/1371438766717/1371216001122/La_UC3M_y_EMT_Madrid_presentan_una_nueva_Catedra_en_Inteligencia_Artificial_para_el_transporte_y_l (accedido: 6 de mayo de 2026).

[9] (Grupo de Inteligencia Artificial Aplicada) https://giaa.uc3m.es/ (accedido: 6 de mayo de 2026).

[10] https://www.crtm.es/ (accedido: 6 de mayo de 2026)

3.2. Estructura de trabajo

El proyecto se puede separar en cuatro partes características. A continuación, se expone cada una con —Nombre, Duración, Explicación de contenido—. En la figura 3.1 se puede ver, representado a modo de diagrama de **Gantt**, las distintas fases espaciadas en el tiempo.

- **Investigación y Planteamiento**: Duración [6 semanas]

  Supone la fase inicial del trabajo. Al no disponer de los datos, expandidos en el capítulo 4, y debido a la naturaleza investigadora del trabajo, en una primera etapa, se realizó un análisis del estado de la cuestión, expuesto en la sección 2, junto con un estudio de la plataforma GAMA [27], con el fin de realizar un acercamiento a los conceptos, estándares y herramientas que se usan a lo largo del desarrollo. Durante esta fase, a lo largo de las reuniones bisemanales, se acota el rango de aplicación del simulador, reduciéndose de las cinco líneas express inicialmente deseadas, a la línea E01, con el fin de no sobrecargar el análisis de la fase posterior.

- **Análisis y Parametrización de Datos**: Duración [12 semanas]

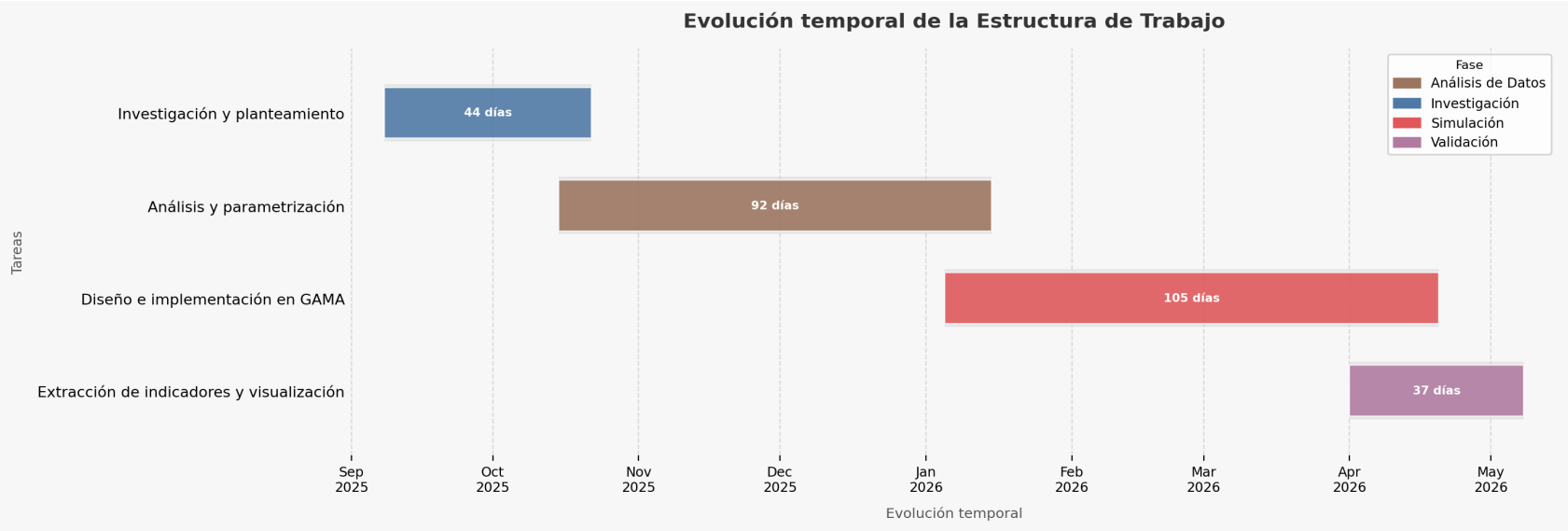
  En esta etapa se realiza un estudio exhaustivo sobre los datos facilitados, tanto los datos geoespaciales de la ruta, como los datos de operación de los autobuses. A lo largo de estas doce semanas, se procesa esta información para analizar la viabilidad de la solución planteada durante la fase uno, basándose en criterios estrictos de modelado de distribuciones para los distintos parámetros que sirven de entrada al simulador.

- **Diseño e Implementación en GAMA**: Duración [14 semanas]

  Una vez estudiada la viabilidad de usar los datos de operación de autobuses y preparados los parámetros de entrada al simulador, en esta etapa se construye la arquitectura del sistema multiagente y su desarrollo en el lenguaje propio de GAMA «GAML» [27]. A lo largo de esta fase, expandida posteriormente, se construyen dos modelos distintos: uno en el que los pasajeros se representan como agentes individuales con comportamiento propio, y otro en el que las subidas y bajadas se modelan de forma agregada por parada, priorizando el rendimiento computacional de la simulación.

- **Extracción de indicadores y visualización**: Duración [5 semanas]

  En esta última fase, se parte del simulador prácticamente completado, y se construye una herramienta de visualización y de lanzamiento en local de los distintos escenarios que plantee el usuario. Por otra parte, se completa el simulador con el feedback de formato de datos que requiere el extractor de KPIs.



Fuente: Elaboración propia.

Fig. 3.1. Diagrama de Gantt explicativo relativo a las distintas fases del desarrollo

3.3. Herramientas Utilizadas

En esta sección se exponen las distintas herramientas utilizadas para la construcción del sistema, junto a una breve explicación de su peso en ella.

- **GAMA**: Usada en el desarrollo del sistema multi-agente, facilita la implementación del estándar de comunicaciones FIPA [20] y la inclusión de mapas geoespaciales.
- **Jupyter Notebook**[11]: Herramienta usada con kernel de *Python* para el análisis de los datos de operación de la línea a simular. Facilita la visualización de resultados, el análisis exploratorio de datos o la construcción de modelos estadísticos. Con su estructura de celdas «markdown» y sus celdas de código, mejora la lectura y la explicabilidad de los procesos que se realizan en cada documento «ipynb» (formato de cuadernos Jupyter).
- **QGIS**[12]: Necesaria para la conversión de GeoJSON (Formato basado en JSON con especificación concreta de parámetros para representación geográfica de los datos) a archivo ESRI shapefile (formato estándar de representación vectorial de datos geoespaciales) . Usada para la carga en el simulador de los datos de la ruta y de la ubicación de las paradas.
- **Node.js**[13]: Framework de desarrollo web utilizado para el despliegue de la herramienta en *local*. Permite la creación de una arquitectura cliente/servidor que permita la conexión entre acciones o deseos del usuario en el navegador y su correspondiente cambio en la máquina anfitriona. Facilita la visualización de las simulaciones o el lanzamiento de nuevas.
- **Docker**[14]: Usado para la estandarización y aislamiento de procesos, facilitando el uso global del sistema. Utilizado para el despliegue de un contenedor encargado de la ejecución del programa completo, emulando un «gemelo digital» con las características del ordenador utilizado para el desarrollo. Gracias a esta herramienta, se permite ejecutar simulaciones en entornos sin la plataforma de desarrollo de agentes instalada.
- **Python**[15]: Lenguaje de programación usado para la extracción de métricas de rendimiento de las distintas simulaciones. Sus librerías diseñadas para análisis de datos sobre grandes volúmenes como puede ser el caso de *pandas* o generación de gráficas concretas con *matplotlib* permiten generar estas KPIs con mayor facilidad.

[11] https://jupyter.org (accedido: 9 de mayo de 2026).
[12] https://qgis.org (accedido: 9 de mayo de 2026).
[13] https://nodejs.org (accedido: 9 de mayo de 2026).
[14] https://www.docker.com (accedido: 9 de mayo de 2026).
[15] https://www.python.org (accedido: 9 de mayo de 2026).

Sobre las herramientas físicas utilizadas, se ha dispuesto de un portátil MSI Modern 15 C13M-273XES, modelo con 16 núcleos lógicos, 32GB RAM DDR4 y almacenamiento NVMe. Esta información, junto a detalles sobre presupuesto, se expanden en el capítulo 8, pero cabe ser destacada por su relación a las herramientas usadas y su interés en la interpretación a nivel de tiempo en la fase de **Diseño e implementación**, así como en la relativa a **Extracción de indicadores y visualización**.

Todo el desarrollo del trabajo se ha realizado sobre el sistema operativo Ubuntu 24.04 LTS, distribución de Linux basada en Debian. Esta elección de Sistema Operativo, sumado al conjunto de herramientas utilizadas, constituye un sistema de **código abierto** en su totalidad. Esto permite un sistema transparente y reproducible, sin dependencia de licencias comerciales.

4. ANÁLISIS DE LOS DATOS

En este capítulo se describe el proceso de análisis y procesado de los datos otorgados por la EMT. Se ha dividido en tres etapas: **Limpieza y Preparación**, **Estudio y Gaussianidad** y **Preparación para Simulador**.

4.1. Limpieza y preparación

Esta sección detalla los distintos procedimientos seguidos en una primera fase iniciada tras la recepción de los datos provenientes de la EMT. Podemos distinguir dos tipos de datos principalmente: Datos Geoespaciales y Datos de Operación.

4.1.1. Datos geoespaciales

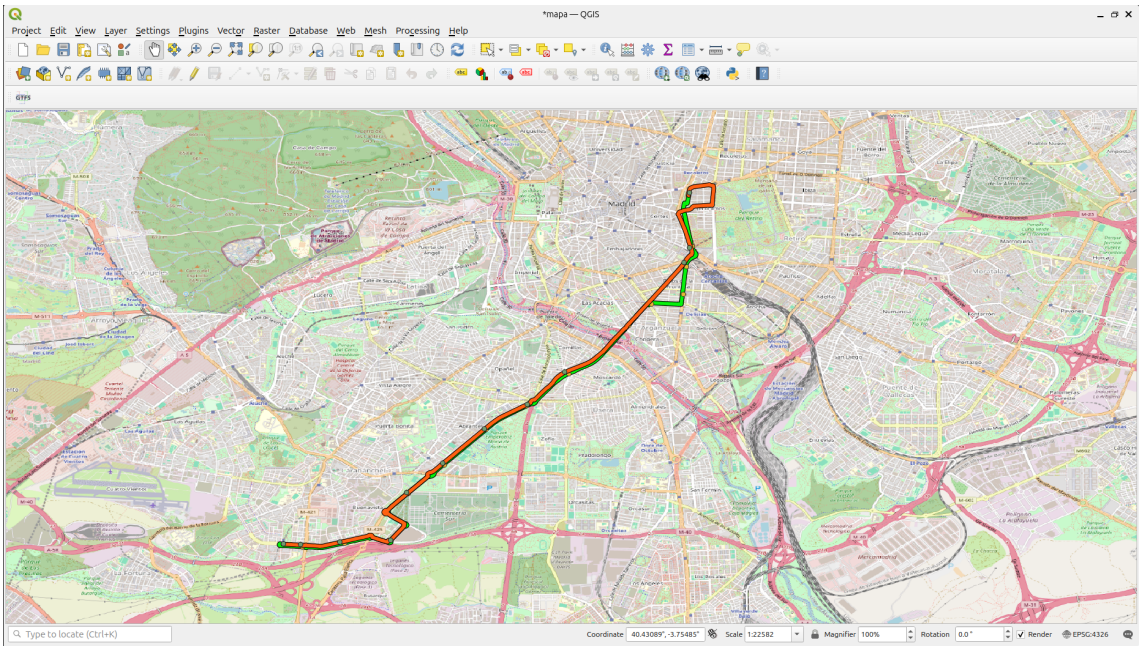
Estos datos se corresponden con elementos públicos obtenidos del **EMT Open Data Portal**[16], una iniciativa de la entidad que busca reforzar su compromiso con la transparencia y la colaboración para la mejoría en la gestión [29]. Si bien el simulador no necesita la exacta localización de las paradas o de la ruta en cuestión, estos datos permiten una mejor visualización de cara a la construcción del modelo. Además, guardan información del orden y código numérico de las paradas, por lo que se impulsó su integración y tratamiento.

Dichos datos se encuentran en formato GeoJSON, un estándar dentro del ámbito geoespacial. No obstante, debido a la ubicación de estos dentro de un fichero de extensión «CSV», se encuentran mal formateados, presentando comillas simples «’ ’» en vez de comillas dobles «""», y juntando ambos sentidos de la ruta dentro de la misma celda.

Para subsanar esto, se recurre a un script de *Python* encargado de modificar las celdas correspondientes a los datos geo, separando en dos ficheros distintos según sentido y cambiando las occurrencias de comillas simples por dobles. Por facilidad, además, se ramificaron según si se correspondian con referencias a la trayectoria de la línea, o a instancias de paradas. De esta forma, se obtienen de cada ruta cuatro archivos, dos por cada sentido, donde uno contiene información de la topología de las uniones inter-parada, y el otro de los puntos referencia a las mismas.

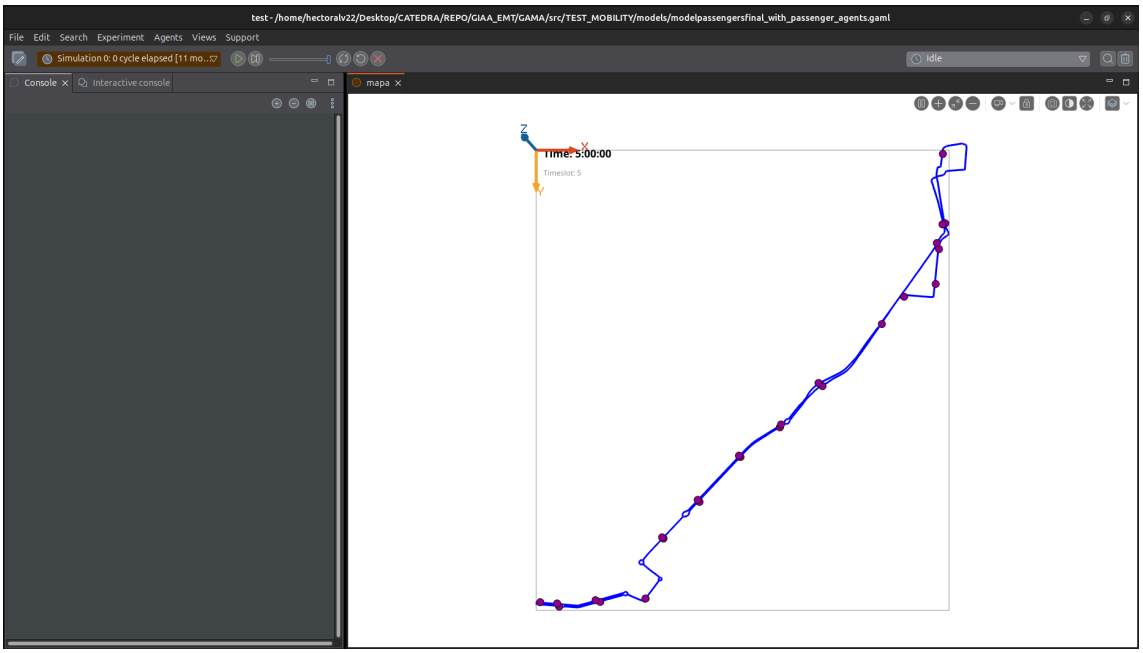
Como se especifica con anterioridad, aparte de la obtención y limpieza de los datos, para facilidad en la implementación, estos pasaron por un proceso de transformación a formato *ESRI shapefile*. Para ello se usa la herramienta de código abierto **QGIS**. En la figura 4.1 se muestra como se visualizan los cuatro GeoJSON procesados y transformados a shapefile. Posteriormente, en la figura 4.2 se muestran esos archivos shapefile ya dentro del entorno de trabajo de la plataforma de agentes GAMA.

[16] https://datos.emtmadrid.es/ (accedido: 9 de mayo de 2026)



Fuente: elaboración propia.

Fig. 4.1. Visualización en QGIS del mapa geoespacial convertido a shapefile



Fuente: elaboración propia.

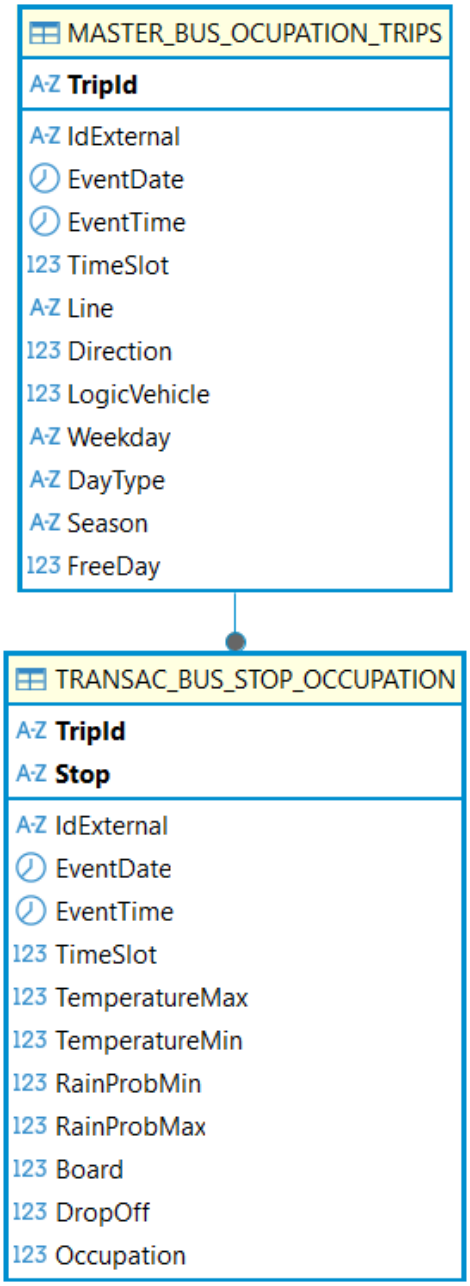
Fig. 4.2. Visualización en GAMA de los shapefile transformados

Estos datos no se usan en las siguientes fases, al no requerir mayor estudio sobre ellos. Se vuelven a abordar en el capítulo 5.

4.1.2. Datos de Operación

Los datos de operación representan la base empírica sobre la que se asienta el simulador. Su tratamiento supuso el inicio formal de la fase dos del proyecto «**Analisis y Parametrización**», comenzada una vez la EMT termina de preparar la base de datos en un servidor dedicado, incluyendo el pipeline que alimenta diariamente los datos. Los geoespaciales por su parte, al ser de acceso público, se pueden tratar sin la necesidad de preparación por parte de la empresa, por lo que se empiezan a tratar dentro de la fase previa a la mencionada.

Sobre la base de datos, en la figura 4.3 se puede ver el esquema relacional. Es un modelo sencillo compuesto por dos tablas, Master y Transaccional, relacionadas por *TripId*, identificador único de cada viaje. De esta forma, cada salida de cabecera representa una instancia de la tabla Master, mientras que cada llegada al resto de paradas de la ruta supone una entrada de la tabla transaccional, identificada por el mismo *TripId* y el código de parada.



Fuente: Documento facilitado por la EMT.

Fig. 4.3. Modelo Master:Transaccional de la base de datos

El resto de parámetros se especifican a continuación.

- **Master::IdExternal**: Identificador interno de la EMT, se especificó no usarlo.
- **Master::EventDate**: Fecha en formato YYYY:MM:DD referente al día del viaje.
- **Master::EventTime**: Hora en formato HH:MM:SS.f representativa del momento de inicio del viaje.

- **Master::TimeSlot**: Franja horaria relativa a Master::EventTime.
- **Master::Line**: Línea del viaje
- **Master::Direction**: Sentido del viaje. De forma interna el valor *1* representaba un sentido y *2* el opuesto.
- **Master::LogicVehicle**: Identificador interno del coche encargado del viaje.
- **Master::Weekday**: Acrónimo referente al día de la semana específico. Puede ser «L,M,X,J,V,S,D» según el día.
- **Master::DayType**: Tipo de día. Existen múltiples valores posibles, principalmente se usa para diferenciar días laborables de fines de semana y festivos.
- **Master::Season**: Época del año relativa al día de recolección del dato.
- **Master::FreeDay**: Especifica si el día presenta *gratuidad* (toma valor «1») o no (valor «0»). El término se refiere a aquellos días específicos en los que el servicio del autobús se presenta gratuito.
- **Transaccional::idExternal**: Mismo caso que en el parámetro de la tabla Master, se ignora.
- **Transaccional::EventDate**: Mismo parámetro que en la tabla Master, salvo que debido a retrasos provoque la llegada al día siguiente en casos extremos al final de la jornada.
- **Transaccional::EventTime**: Hora en igual formato que Master::EventTime. Almacena el momento exacto en el que el bus llega a la parada específica.
- **Transaccional::TimeSlot**: Franja horaria relativa al parámetro Transaccional::EventTime. Puede diferir del campo Master::TimeSlot.
- **Transaccional::TemperatureMax**: Previsión máxima de temperatura en la parada en el momento de la recolección del dato.
- **Transaccional::TemperatureMin**: Mismo caso que Transaccional::TemperatureMax pero para la previsión mínima
- **Transaccional::RainProbMax**: Valor máximo de probabilidad de precipitación registrado en el instante que se guarda la instancia.
- **Transaccional::RainProbMin**: Valor mínimo de probabilidad de precipitación en el mismo instante.
- **Transaccional::Board**: Número de pasajeros que carga el autobús en el valor de la bitupla TripId-Stop.

- **Transaccional::DropOff**: Número de pasajeros que abandonan el autobús.
- **Transaccional::Occupation**: Valor de la carga de pasajeros derivada del cálculo $Transaccional :: Occupation_{prev}$ + Transaccional::Board - Transaccional::DropOff.

El estudio ha derivado de distintas consultas sobre estas tablas, que encapsulaban en un único fichero «CSV» toda la información referente a una parada. En el código 4.1 se muestra el código *SQL* utilizado para las consultas sobre la base de datos.

CÓDIGO 4.1. Datos totales de una parada

```
SELECT mbot.*, tbso.*
FROM UCIII.Transport.MASTER_BUS_OCUPATION_TRIPS mbot
JOIN UCIII.Transport.TRANSAC_BUS_STOP_OCCUPATION tbso
    ON tbso.TripId = mbot.TripId
    WHERE tbso.Stop = 'STOPCODE';
```

Fuente: Documento Facilitado por la EMT.

El resultante «CSV» presentaba derivado del *JOIN* un problema, y era la duplicidad de nomenclatura en campos de ambas tablas, en concreto —*IdExternal*, *EventDate*, *EventTime* y *TimeSlot*—.

Como se ha mencionado, el análisis de los datos de operación se ha realizado con la herramienta **Jupyter Notebook**, usando kernel de *Python*. Bajo este framework, para el manejo de los volúmenes de datos se ha usado la librería *pandas*, que ante duplicidad en los nombres de los campos, a la segunda ocurrencia añade un «.1» al cuerpo de la columna. De esta forma, todos aquellos campos contenedores de un «.1» se corresponden con las referentes a la tabla Transaccional. De aquí en adelante, no obstante, para diferenciar ambos parámetros, aparecerá el prefijo *Transaccional::* o *Master::* para no generar ambigüedades.

En lo relativo a la limpieza de los datos, se trata de una fase clave, porque la no identificación de posibles anomalías en ciertos campos puede alterar la precisión del análisis posterior. En el caso particular, se redujo la limpieza a la eliminación de valores anómalos dentro de los campos *Transaccional::Occupation*, *Transaccional::Board* y *Transaccional::DropOff*, debido a la identificación de inconsistencias relativas a valores negativos en estos campos, o a valores extremos derivados de errores en la recolección. Como se comenta de forma exhaustiva en la siguiente sección, estos campos se presentan como los más importantes del análisis, motivo por el que se tratan de forma exhaustiva para evitar problemas.

4.2. Estudio y Gaussianidad

4.3. Preparación para Simulador

5. DISEÑO DE LA SIMULACIÓN

6. CASOS DE APLICACIÓN

7. MARCO REGULADOR

8. PRESUPUESTO Y COSTES

9. TRABAJOS FUTUROS

10. CONCLUSIÓN

BIBLIOGRAFÍA

[1] M.-P. Pelletier, M. Trépanier y C. Morency, "Smart card data use in public transit: A literature review," *Transportation Research Part C: Emerging Technologies*, vol. 19, n.º 4, pp. 557-568, 2011.

[2] S. Nesmachnow y D. Hernández, "Big Data Analysis for Travel Time Characterization in Public Transportation Systems," *Sustainability*, vol. 15, n.º 19, p. 14 561, 2023.

[3] P. Maes, "Modeling Adaptive Autonomous Agents," *Artificial Life*, vol. 1, n.º 1–2, pp. 135-162, 1993.

[4] N. R. Jennings y M. J. Wooldridge, "Applications of Intelligent Agents," en *Agent Technology: Foundations, Applications, and Markets*, N. R. Jennings y M. J. Wooldridge, eds., Berlin, Heidelberg: Springer-Verlag, 1998, pp. 3-28.

[5] H. S. Nwana, "Software Agents: An Overview," *The Knowledge Engineering Review*, vol. 11, n.º 3, pp. 205-244, 1996.

[6] M. Wooldridge y N. R. Jennings, "Intelligent Agents: Theory and Practice," *The Knowledge Engineering Review*, vol. 10, n.º 2, pp. 115-152, 1995.

[7] C. Castelfranchi, "Guarantees for Autonomy in Cognitive Agent Architecture," en *Intelligent Agents: Theories, Architectures, and Languages (LNAI Volume 890)*, M. Wooldridge y N. R. Jennings, eds., Springer-Verlag, 1995, pp. 56-70.

[8] R. A. Brooks, "A Robust Layered Control System for a Mobile Robot," *IEEE Journal on Robotics and Automation*, vol. 2, n.º 1, pp. 14-23, 1986.

[9] S. Franklin y A. Graesser, "Is It an Agent, or Just a Program? A Taxonomy for Autonomous Agents," en *Intelligent Agents III (LNAI Volume 1193)*, J. P. Müller, M. J. Wooldridge y N. R. Jennings, eds., Springer-Verlag, 1997, pp. 21-35.

[10] M. E. Bratman, *Intention, Plans, and Practical Reason*. Cambridge, MA: Harvard University Press, 1987.

[11] R. A. Brooks, "Elephants Don't Play Chess," en *Designing Autonomous Agents*, P. Maes, ed., Cambridge, MA: MIT Press, 1990, pp. 3-15.

[12] R. A. Brooks, "Intelligence Without Reason," en *Proceedings of the Twelfth International Joint Conference on Artificial Intelligence (IJCAI-91)*, Sydney, Australia, 1991, pp. 569-595.

[13] R. A. Brooks, "Intelligence Without Representation," *Artificial Intelligence*, vol. 47, pp. 139-159, 1991.

[14] I. A. Ferguson, "TouringMachines: An Architecture for Dynamic, Rational, Mobile Agents," Tesis doct., University of Cambridge, 1992.

[15] J. P. Müller, M. Pischel y M. Thiel, "Modeling Reactive Behaviour in Vertically Layered Agent Architectures," en *Intelligent Agents (LNAI Volume 890)*, M. Wooldridge y N. R. Jennings, eds., Springer-Verlag, 1995, pp. 261-276.

[16] E. Werner, "Logical Foundations of Distributed Artificial Intelligence," en *Foundations of Distributed Artificial Intelligence*, G. Weiss, ed., John Wiley & Sons, 1996, pp. 57-99.

[17] H. V. D. Parunak, "Applications of Distributed Artificial Intelligence in Industry," en *Foundations of Distributed Artificial Intelligence*, G. M. P. O'Hare y N. R. Jennings, eds., Wiley Inter-Science, 1996, pp. 139-164.

[18] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2.ª ed. Chichester, UK: John Wiley & Sons, 2009.

[19] V. Botti, "Agentic AI and Multiagentic: Are We Reinventing the Wheel?" *arXiv preprint arXiv:2506.01463*, 2025.

[20] Foundation for Intelligent Physical Agents, *FIPA ACL Message Structure Specification*, FIPA Specification SC00061G, 2002. [En línea]. Disponible en: http://www.fipa.org/specs/fipa00061/.

[21] T. Finin, R. Fritzson, D. McKay y R. McEntire, "KQML as an Agent Communication Language," en *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM-94)*, ACM, 1994, pp. 456-463.

[22] Foundation for Intelligent Physical Agents, *FIPA Communicative Act Library Specification*, FIPA Specification SC00037J, 2002. [En línea]. Disponible en: http://www.fipa.org/specs/fipa00037/.

[23] F. L. Bellifemine, G. Caire y D. Greenwood, *Developing Multi-Agent Systems with JADE*. Chichester, UK: John Wiley & Sons, 2007.

[24] U. Wilensky, *NetLogo*, Center for Connected Learning and Computer-Based Modeling, Northwestern University, 1999. [En línea]. Disponible en: http://ccl.northwestern.edu/netlogo/.

[25] S. Luke, C. Cioffi-Revilla, L. Panait, K. Sullivan y G. Balan, "MASON: A Multi-agent Simulation Environment," *Simulation*, vol. 81, n.º 7, pp. 517-527, 2005.

[26] R. H. Bordini, J. F. Hübner y M. Wooldridge, *Programming Multi-Agent Systems in AgentSpeak Using Jason*. John Wiley & Sons, 2007.

[27] P. Taillandier et al., "Building, composing and experimenting macroscopic models with the GAMA platform," *GeoInformatica*, vol. 23, n.º 2, pp. 299-322, 2019. [En línea]. Disponible en: https://gama-platform.org.

[28] M. Saidallah, A. El Fergougui y A. E. Elalaoui, "A Comparative Study of Urban Road Traffic Simulators," *MATEC Web of Conferences*, vol. 81, p. 05 002, 2016.

[29] S. Balaguer, M. Fernández y A. Martín, "Data as an asset: EMT open platform for transport data in Madrid," *Collection of Open Conferences in Research Transport*, vol. 2018, 2018. DOI: 10.5281/zenodo.1441026.